Звіт з індивідуального завдання з дисципліни «Технології DevOps»

на тему: «Реалізація інфраструктури DevOps для забезпечення оновлення та підтримання актуальності програмного забезпечення сервера „Розумного будинку“ на базі апаратної платформи Raspberry Pi з підтримкою інтеграції цифрового двійника»

2026

Реалізація інфраструктури DevOps для забезпечення оновлення та підтримання актуальності програмного забезпечення сервера «Розумного будинку» на базі апаратної платформи Raspberry Pi з підтримкою інтеграції цифрового двійника

Зайкін Андрій Вікторович,
здобувач другого курсу магістратури, групи КІ-25-1м,
кафедри комп'ютерної інженерії та електроніки,
Кременчуцький національний університет імені Михайла Остроградського

Дружинін Валерій Олегович,
асистент кафедри комп'ютерної інженерії та електроніки,
Кременчуцький національний університет імені Михайла Остроградського

АНОТАЦІЯ

Робота присвячена проєктуванню інфраструктури DevOps для сервера «Розумного будинку» на базі Raspberry Pi 4 (2 ГБ) з СКБД MySQL, веб-інтерфейсом налаштування датчиків і статистики, функціями керування кондиціонуванням та опаленням і мікросервісним API для цифрового двійника. Об'єктом дослідження є процес розроблення, доставки та супроводу програмного забезпечення сервера, а предметом – інфраструктура DevOps для його оновлення та підтримання актуальності.

Метою роботи є створення відтворюваного конвеєра безперервної інтеграції та доставки, який забезпечує регулярне й безпечне оновлення ПЗ сервера з автоматичним відкатом у разі збою. У роботі обґрунтовано актуальність DevOps для проєкту, розподілено зони відповідальності за матрицею RACI, визначено 12 задач впровадження, описано життєвий цикл ПЗ, обрано інструментальні засоби, сумісні з архітектурою ARM64 (GitHub Actions, Docker, Ansible, Prometheus/Grafana), і сплановано витрати часу: 232 год очікуваної трудомісткості та 246 год планового бюджету за 10 тижнів.

Результати можуть бути використані для інших науково-технічних проєктів кафедри, побудованих навколо «Розумного будинку» та мікроконтролерів ESP8266.

Ключові слова: DevOps, CI/CD, Raspberry Pi, розумний будинок, цифровий двійник, Docker, MQTT, IoT.

ЗМІСТ

ВСТУП.....4
1 АКТУАЛЬНІСТЬ ІНТЕГРАЦІЇ ІНФРАСТРУКТУРИ DEVOPS У НАУКОВО-ТЕХНІЧНИЙ ПРОЄКТ.....5
2 РОЗПОДІЛ ЗОН ВІДПОВІДАЛЬНОСТІ ПРИ ВПРОВАДЖЕННІ ІНФРАСТРУКТУРИ DEVOPS.....7
3 ВИЗНАЧЕННЯ ЗАДАЧ ДЛЯ ВПРОВАДЖЕННЯ ІНФРАСТРУКТУРИ DEVOPS.....9
4 ОПИС ЖИТТЄВОГО ЦИКЛУ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ.....11
5 ПЛАНУВАННЯ ЗАСОБІВ ДЛЯ СТВОРЕННЯ СЕРЕДОВИЩА DEVOPS.....13
6 ПЛАНУВАННЯ ВИТРАТ ЧАСУ НА ІНТЕГРАЦІЮ ІНФРАСТРУКТУРИ DEVOPS.....16
ВИСНОВКИ.....18
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ.....19

ВСТУП

З переліку актуальних науково-технічних досліджень кафедри КІЕ обрано тему № 4 «Сервер „Розумного будинку“ на базі апаратної платформи Raspberry Pi з підтримкою інтеграції цифрового двійника». Серед усіх тем переліку ця найбільше насичена програмним забезпеченням: сервер є центральним вузлом, до якого підключаються «розумні датчики» на ESP8266 (теми № 3, 5, 6), а тому частота його змін і вимоги до надійності найвищі.

Вихідні дані теми: апаратна платформа Raspberry Pi 4 (2 ГБ ОЗП); СКБД MySQL для зберігання показів датчиків; окрема сторінка налаштування датчиків і окрема – статистики; збереження даних за запитом сервера; базові функції керування кондиціонуванням і опаленням; програмні функції для мікросервісів керування через цифровий двійник.

Мета роботи – сформувати бачення інфраструктури DevOps, що забезпечить оновлення й підтримання актуальності проєктованого ПЗ сервера.

Для досягнення мети поставлено такі **задачі**:

1) обґрунтувати актуальність інтеграції DevOps у проєкт;
2) розподілити зони відповідальності між учасниками;
3) визначити задачі впровадження;
4) описати життєвий цикл ПЗ;
5) спланувати інструментальні засоби середовища DevOps;
6) спланувати витрати часу на інтеграцію.

1 АКТУАЛЬНІСТЬ ІНТЕГРАЦІЇ ІНФРАСТРУКТУРИ DEVOPS У НАУКОВО-ТЕХНІЧНИЙ ПРОЄКТ

DevOps потрібен цьому проєкту тому, що сервер «Розумного будинку» працює безперервно, керує фізичним обладнанням (опалення, кондиціонування) і при цьому постійно змінюється в ході досліджень. Ручне копіювання файлів на Raspberry Pi через SSH та ручні зміни схеми MySQL призводять до невідтворюваних станів системи й втрат даних експериментів.

Основні чинники актуальності:

– **висока частота змін** – у науковому проєкті регулярно з'являються нові типи датчиків (DHT11, MQ-2, MQ-135, MQ-5, BMP180, DS18B20), нові алгоритми керування та версії моделі цифрового двійника; кожна зміна має доставлятися швидко й без зупинки збору даних;

– **розподілена архітектура** – система складається з кількох мікросервісів (прийом даних, веб-інтерфейс, керування кліматом, API цифрового двійника), СКБД та MQTT-брокера, їх версії мають бути узгоджені;

– **обмежена апаратна платформа** – Raspberry Pi 4 з 2 ГБ ОЗП і картою microSD чутливий до збоїв живлення й зносу носія; автоматизоване розгортання з коду (Infrastructure as Code) дозволяє відновити сервер за хвилини, а не години;

– **відтворюваність наукових результатів** – версіонування коду, конфігурації та міграцій БД дозволяє точно вказати, яка версія ПЗ зібрала конкретний набір даних чи дала конкретний результат оптимізації;

– **безпека IoT** – сервер доступний у мережі й керує реальними пристроями, тому регулярне оновлення залежностей і автоматичне сканування вразливостей є обов'язковими;

– **наступність команди** – у студентських проєктах склад виконавців змінюється щороку; автоматизовані конвеєри й документована інфраструктура забезпечують передачу проєкту наступному поколінню дослідників.

Дослідження DORA протягом десяти років показують, що надійність і швидкість доставки ПЗ вимірюються чотирма ключовими метриками: час від

зміни до розгортання, частота розгортань, частка невдалих змін і час відновлення після збою [1, 2]. Ці ж метрики прийнято цільовими для проєкту (таблиця 1.1).

Таблиця 1.1 – Цільові показники DevOps для проєкту

Метрика	**Без DevOps (ручне розгортання)**	**Ціль після впровадження**
Час від коміту до роботи на сервері	1–3 дні	≤ 30 хв
Частота розгортань	раз на 1–2 тижні	за кожним злиттям у main (кілька разів на тиждень)
Частка невдалих змін	не вимірюється	≤ 15 %
Час відновлення після збою	години (перевстановлення вручну)	≤ 15 хв (відкат на попередній образ)

Таким чином, інтеграція DevOps перетворює сервер з «унікального стенда» на відтворювану платформу, яку можна розвивати та передавати в інші проєкти кафедри (теми № 1, 7, 9 також побудовані навколо «Розумного будинку» [3]).

2 РОЗПОДІЛ ЗОН ВІДПОВІДАЛЬНОСТІ ПРИ ВПРОВАДЖЕННІ ІНФРАСТРУКТУРИ DEVOPS У НАУКОВО-ТЕХНІЧНИЙ ПРОЄКТ

У невеликій науковій команді одна людина може виконувати кілька ролей, тому відповідальність розподілено за ролями, а не за посадами. Для фіксації розподілу використано матрицю RACI: R – виконує, A – затверджує й несе кінцеву відповідальність, C – консультує, I – інформується.

Ролі учасників проєкту:

– **НК – науковий керівник** (власник продукту): формує вимоги, затверджує релізи та план експериментів;

– **РС – розробник серверного ПЗ** (виконавець теми): мікросервіси, веб-інтерфейс, схема БД, API цифрового двійника;

– **DO – DevOps-інженер** (у цьому проєкті поєднується з РС): конвеєр CI/CD, контейнеризація, IaC, моніторинг, резервне копіювання;

– **РВ – розробники вбудованого ПЗ** (виконавці суміжних тем № 3, 5, 6 на ESP8266): формат повідомлень MQTT, сумісність прошивок із сервером;

– **ТД – тестувальник-дослідник**: приймальні та натурні випробування на стенді, валідація даних;

– **АК – адміністратор мережі кафедри**: доступ до мережі, VPN, правила міжмережевого екрану.

Таблиця 2.1 – Матриця відповідальності RACI

Вид діяльності	НК	РС	DO	РВ	ТД	АК
Формування вимог і беклогу	A	R	C	C	C	I
Розроблення коду сервера та міграцій БД	I	A/R	C	C	I	–
Перегляд коду (code review)	C	R	A	C	I	–
Конвеєр CI/CD, збірка образів	I	C	A/R	I	I	–
Автоматичні тести	I	R	A	C	C	–
Контракт MQTT «датчик – сервер»	I	A	C	R	C	–
Підготовка Raspberry Pi та IaC	I	C	A/R	–	I	C
Мережа, VPN, доступ ззовні	I	I	C	–	–	A/R
Моніторинг і сповіщення	I	C	A/R	I	C	I

Вид діяльності	НК	РС	DO	РВ	ТД	АК
Резервне копіювання даних експериментів	C	I	A/R	–	I	C
Приймальні випробування на стенді	A	C	C	C	R	–
Затвердження релізу	A	R	C	I	C	I
Реагування на інциденти	I	C	A/R	C	I	C

Ключовий принцип розподілу – «хто створив, той і супроводжує»: розробник серверного ПЗ також отримує сповіщення моніторингу й бере участь у розборі інцидентів [4]. Щоб поєднання ролей РС і DO в одній особі не порушувало контролю, злиття в гілку main дозволене лише після успішного проходження CI та схвалення щонайменше одним іншим учасником (НК або РВ).

3 ВИЗНАЧЕННЯ ЗАДАЧ ДЛЯ ВПРОВАДЖЕННЯ ІНФРАСТРУКТУРИ DEVOPS У НАУКОВО-ТЕХНІЧНИЙ ПРОЄКТ

Впровадження розбито на 12 задач, згрупованих у чотири напрями: керування кодом, безперервна інтеграція (CI), безперервна доставка (CD), експлуатація. Кожна задача має перевірюваний критерій виконання (таблиця 3.1).

Таблиця 3.1 – Задачі впровадження інфраструктури DevOps

№	Напрям	Задача	Критерій виконання
ЗД-1	Керування кодом	Створити репозиторій (монорепозиторій сервера, окремий – для прошивок), правила гілок (GitHub Flow), шаблони issue/PR	Захищена гілка main, злиття лише через PR
ЗД-2	Керування кодом	Ввести семантичне версіонування та Conventional Commits, автоматичний CHANGELOG	Тег vX.Y.Z створює реліз з переліком змін
ЗД-3	Керування кодом	Версіонувати схему MySQL через міграції (Alembic)	БД створюється з нуля однією командою
ЗД-4	CI	Налаштувати конвеєр: лінтер, модульні тести, покриття	Кожен PR перевірено за ≤ 10 хв, покриття ≥ 70 %
ЗД-5	CI	Інтеграційні тести з MySQL і MQTT-брокером у контейнерах, емулятор «розумних датчиків»	Сценарій «показ → БД → статистика → команда на реле» проходить автоматично
ЗД-6	CI	Контейнеризувати мікросервіси, збирати образи linux/arm64	Образи публікуються в реєстр з тегом версії
ЗД-7	CI	Сканування залежностей і образів на вразливості, автооновлення залежностей	У main немає вразливостей рівня High/Critical
ЗД-8	CD	Описати підготовку Raspberry Pi як код (Ansible): ОС, Docker, користувачі, екран, SSH	Чиста плата готова до роботи за ≤ 30 хв
ЗД-9	CD	Автоматичне розгортання на стенд (staging) та ручно затверджуване – на робочий сервер з відкатом	Відкат на попередню версію ≤ 15 хв
ЗД-10	CD	Управління секретами (паролі БД, MQTT, токени) поза кодом	У репозиторії жодного секрету у відкритому вигляді
ЗД-11	Експлуатація	Моніторинг сервера, сервісів і датчиків; централізовані журнали; сповіщення в Telegram	Збій сервісу або «зникнення» датчика виявляється за ≤ 5 хв

№	Напрям	Задача	Критерій виконання
ЗД-12	Експлуатація	Резервне копіювання MySQL і конфігурацій, перевірка відновлення	Щодобова копія поза Raspberry Pi; відновлення перевірене щомісяця

Задачі ЗД-1…ЗД-3 є передумовою для решти. ЗД-6 блокує ЗД-9, а ЗД-8 має бути виконана до першого автоматичного розгортання. Залежності враховано в календарному плані розділу 6.

4 ОПИС ЖИТТЄВОГО ЦИКЛУ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ У НАУКОВО-ТЕХНІЧНОМУ ПРОЄКТІ

Для проєкту обрано ітеративно-інкрементну модель життєвого циклу з двотижневими ітераціями, усередині яких зміни проходять безперервний цикл DevOps. На верхньому рівні життєвий цикл узгоджено зі стадіями ISO/IEC/IEEE 12207:2017, який поділяє процеси на договірні, організаційно-забезпечувальні, технічного управління та технічні [5]. Інфраструктура DevOps реалізує процес «управління інфраструктурою» та автоматизує процеси інтеграції, верифікації, передачі та супроводу за принципами безперервної доставки [11]. Стадії життєвого циклу наведено в таблиці 4.1.

Таблиця 4.1 – Стадії життєвого циклу ПЗ сервера «Розумного будинку»

Стадія	**Зміст у проєкті**	**Результати (артефакти)**	**Підтримка DevOps**
1. Задум і вимоги	Перелік датчиків, сценарії керування кліматом, вимоги цифрового двійника	Беклог (GitHub Issues), контракт MQTT, специфікація OpenAPI	Шаблони issue, дошка GitHub Projects
2. Проєктування	Мікросервісна архітектура, схема БД, інтерфейси між сервісами	Діаграми архітектури, ADR (записи архітектурних рішень)	Документація як код у репозиторії
3. Розроблення	Код мікросервісів, веб-сторінок, міграцій БД	Гілки задач, PR	Локальне середовище Docker Compose з емулятором датчиків, pre-commit
4. Інтеграція та верифікація	Модульні, інтеграційні та контрактні тести	Звіти тестів і покриття, образи контейнерів	GitHub Actions, сканування вразливостей
5. Передача (випуск)	Розгортання на стенд, приймальні випробування, реліз	Реліз vX.Y.Z, CHANGELOG, протокол випробувань	CD на staging, затвердження релізу НК
6. Експлуатація	Збір даних експериментів, керування кліматом, обмін із цифровим двійником	Дані в MySQL, журнали, метрики	Моніторинг, сповіщення, резервні копії

Стадія	Зміст у проєкті	Результати (артефакти)	Підтримка DevOps
7. Супровід	Виправлення дефектів, оновлення залежностей і ОС, підключення нових датчиків	Hotfix-релізи, звіти про інциденти	Dependabot, повторний запуск Ansible, відкат
8. Виведення з експлуатації	Завершення дослідження або передача наступній команді	Архів даних, архівований репозиторій, інструкція розгортання	Відтворення середовища з коду

У проєкті передбачено три середовища з однаковими образами контейнерів:

– **dev** – ноутбук розробника: Docker Compose, MySQL, MQTT-брокер і емулятор датчиків, що публікує синтетичні покази;

– **staging** – стендовий Raspberry Pi з 1–2 реальними датчиками ESP8266, розгортається автоматично з main;

– **production** – робочий Raspberry Pi 4 у лабораторії, розгортається з тегу релізу після затвердження.

Оновлення прошивок ESP8266 не входить у цей життєвий цикл, але межа між ними зафіксована версіонованим контрактом MQTT: сервер підтримує поточну та попередню версії формату повідомлень, тож датчики можна оновлювати незалежно.

5 ПЛАНУВАННЯ ЗАСОБІВ ДЛЯ СТВОРЕННЯ СЕРЕДОВИЩА DEVOPS У НАУКОВО-ТЕХНІЧНОМУ ПРОЄКТІ

Засоби обрано за трьома критеріями: безкоштовність для навчального проєкту, підтримка архітектури ARM64 (Raspberry Pi 4) та помірне споживання пам'яті в межах 2 ГБ ОЗП. Результат наведено в таблиці 5.1.

Таблиця 5.1 – Засоби середовища DevOps

Категорія	Засіб	Призначення в проєкті	Обґрунтування вибору
Керування версіями	Git, GitHub	Код, IaC, документація, issue, PR, релізи	Безкоштовно для студентів, вбудовані CI та реєстр
Планування	GitHub Projects, Issues	Беклог, дошка ітерацій	Зв'язок задач із комітами та PR
Серверне ПЗ	Python 3.12, FastAPI, SQLAlchemy, Alembic	Мікросервіси, REST API для цифрового двійника, міграції БД	Автоматична специфікація OpenAPI, низьке споживання ресурсів
Дані та обмін	MySQL 8, Eclipse Mosquitto	Зберігання показів; MQTT-брокер для ESP8266	MySQL – умова завдання; Mosquitto – легкий брокер MQTT 5.0/3.1.1 [6]
Тестування і якість	pytest, Testcontainers, Ruff, pre-commit	Модульні й інтеграційні тести, статичний аналіз	Інтеграційні тести зі справжніми MySQL і Mosquitto у контейнерах
CI/CD	GitHub Actions; self-hosted runner на Raspberry Pi	Збірка, тести, публікація образів, розгортання	Власний runner сам забирає задачі з GitHub, вхідні порти в мережі кафедри не потрібні [7]
Контейнеризація	Docker Engine, Docker Compose, Buildx	Ізоляція сервісів, однакове середовище dev/staging/production	Buildx збирає образи linux/arm64 на x86-машинах CI [8]
Реєстр артефактів	GitHub Container Registry (GHCR)	Зберігання версіонованих образів	Відкат = запуск попереднього тегу образу
Інфраструктура як код	Ansible, Ansible Vault	Підготовка Raspberry Pi OS (64-bit), Docker, користувачі, ufw, SSH; секрети	Без агента на платі, працює через SSH [9]
Безпека	Dependabot, Trivy, GitHub Secrets	Оновлення залежностей, сканування образів, зберігання токенів	Підтримання актуальності та безпеки залежностей

Категорія	Засіб	Призначення в проєкті	Обґрунтування вибору
Моніторинг	Prometheus, node_exporter, mysqld_exporter, Grafana	Метрики плати (CPU, температура, диск), сервісів і часу останнього показу кожного датчика	Відкритий стек; Grafana й сховище метрик винесено на сервер кафедри [10]
Журнали і сповіщення	Grafana Loki + Alloy, Alertmanager → Telegram	Централізовані журнали, сповіщення про збої	Повідомлення на телефон відповідального
Резервне копіювання	mysqldump + restic	Щодобові зашифровані копії БД і конфігурацій на сервер кафедри	Дедуплікація, шифрування, перевірка відновлення
Вбудоване ПЗ (суміжне)	PlatformIO у GitHub Actions	Збірка прошивок ESP8266 та перевірка контракту MQTT	Єдиний підхід до CI для сервера й датчиків

Апаратне забезпечення середовища: робочий Raspberry Pi 4 (2 ГБ) із завантаженням Raspberry Pi OS (64-bit) [12] з USB-SSD замість microSD (ресурс запису для БД), другий Raspberry Pi або віртуальна машина ARM64 для staging, сервер кафедри для моніторингу й резервних копій, джерело безперебійного живлення для захисту від відключень електроенергії.

Конвеєр CI/CD працює так (рисунок 5.1): кожен PR проходить лінтер і тести на хмарних ранерах GitHub; після злиття в main збираються й скануються образи arm64, вони публікуються в GHCR і автоматично розгортаються на staging; тег релізу після затвердження науковим керівником розгортає ту саму версію на production. Після розгортання виконується перевірка стану (health check); якщо вона невдала, виконується автоматичний відкат на попередній тег образу. Перед застосуванням міграцій БД автоматично робиться резервна копія. Конфігураційні файли конвеєра (GitHub Actions), скрипт розгортання з відкатом і опис сервісів Docker Compose подано в електронних додатках до звіту.

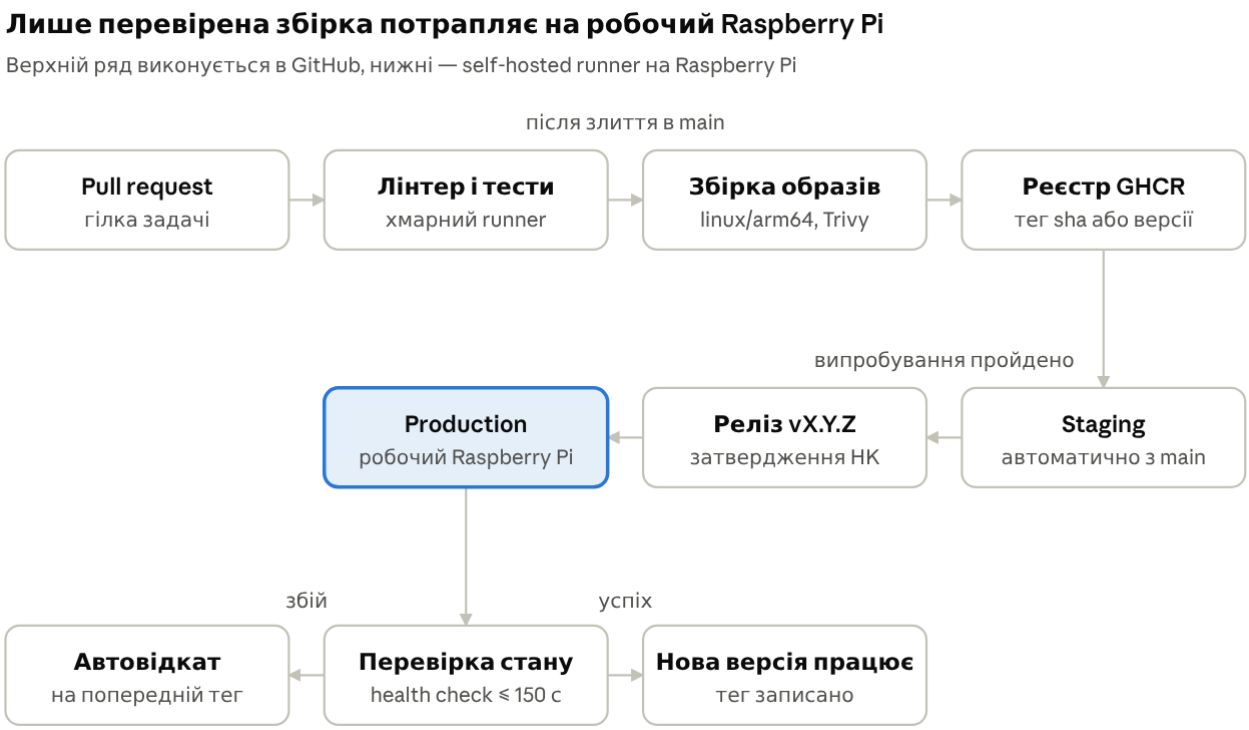


Рисунок 5.1 – Конвеєр CI/CD сервера «Розумного будинку»

6 ПЛАНУВАННЯ ВИТРАТ ЧАСУ НА ІНТЕГРАЦІЮ ІНФРАСТРУКТУРИ DEVOPS У НАУКОВО-ТЕХНІЧНИЙ ПРОЄКТ

Очікувана трудомісткість інтеграції становить 232 людино-години, а з урахуванням резерву на ризики плановий бюджет – 246 год за 10 тижнів (≈ 25 год/тиждень, що відповідає зайнятості магістранта на неповний робочий день). Оцінки отримано методом PERT за трьома значеннями для кожної задачі: оптимістичним (O), найімовірнішим (M) і песимістичним (P):

$$T_e = (O + 4M + P) / 6, \quad \sigma = (P - O) / 6.$$

Таблиця 6.1 – Оцінка трудомісткості задач (людино-години)

№	Задача	O	M	P	Te	Тижні
ЗД-1	Репозиторій, правила гілок, шаблони	6	8	12	8,3	1
ЗД-2	Версіонування, CHANGELOG	4	6	10	6,3	1
ЗД-3	Міграції схеми MySQL	10	14	20	14,3	1–2
ЗД-4	Базовий CI: лінтер, модульні тести	12	16	22	16,3	2–3
ЗД-5	Інтеграційні тести, емулятор датчиків	24	32	44	32,7	3–5
ЗД-6	Контейнеризація, образи arm64	16	24	36	24,7	3–4
ЗД-7	Сканування вразливостей, Dependabot	6	8	12	8,3	4
ЗД-8	Підготовка Raspberry Pi через Ansible	18	24	34	24,7	4–5
ЗД-10	Управління секретами	6	8	12	8,3	5
ЗД-9	CD на staging/production, відкат	20	28	40	28,7	6–7
ЗД-11	Моніторинг, журнали, сповіщення	20	28	40	28,7	7–8
ЗД-12	Резервне копіювання та відновлення	10	14	20	14,3	8–9
–	Документація, передача знань команді	12	16	22	16,3	9–10
	Усього	164	226	324	232,0	10

Стандартне відхилення сумарної оцінки – $\sqrt{\Sigma\sigma^2} \approx 8{,}2$ год. Щоб вкластися в бюджет із ймовірністю 95 %, додано резерв $1{,}65\sigma \approx 14$ год: 232 + 14 = 246 год. Резерв покриває основні ризики: повільна збірка образів arm64 через емуляцію QEMU, несумісність залежностей з ARM64, відмова microSD/SSD та відключення електроенергії.

Критичний шлях: ЗД-1 → ЗД-3 → ЗД-4 → ЗД-6 → ЗД-9 → ЗД-11 → ЗД-12 → документація. Затримка будь-якої з цих задач зсуває завершення

впровадження; задачі ЗД-5, ЗД-7, ЗД-8 і ЗД-10 мають запас часу та виконуються паралельно (рисунок 6.1).

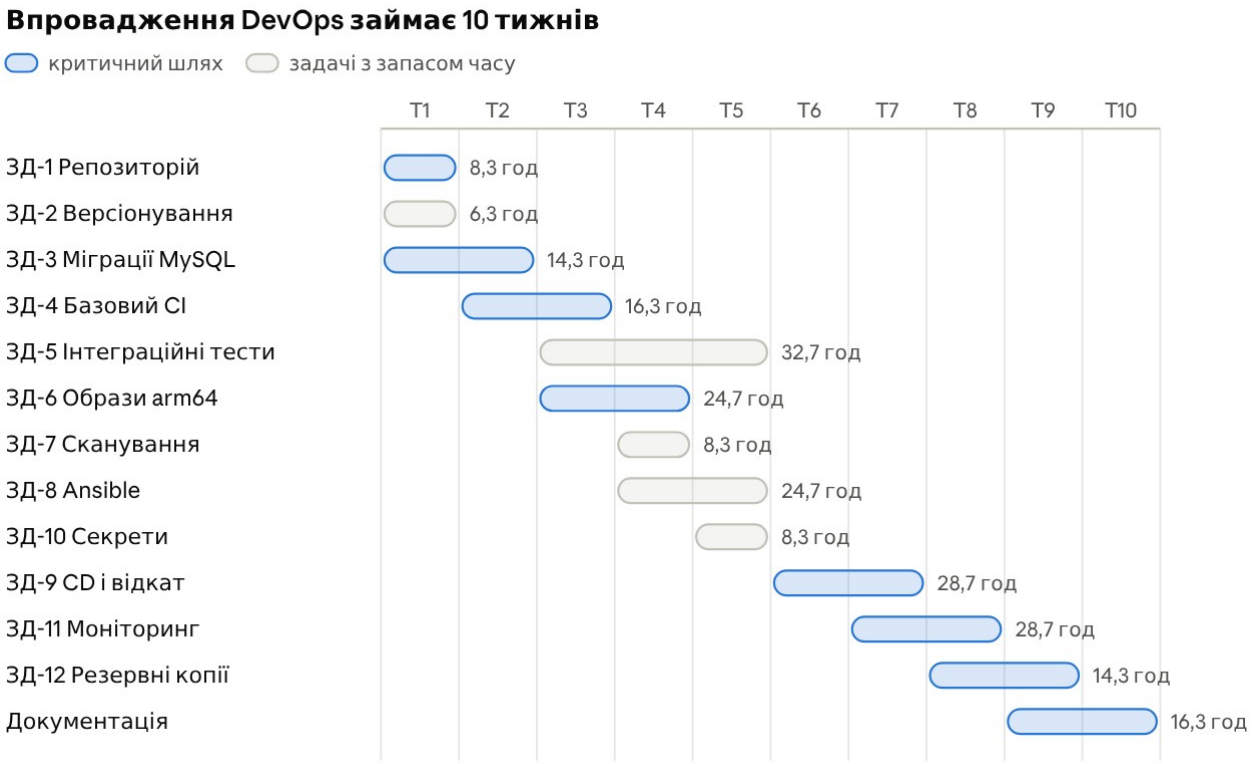


Рисунок 6.1 – Календарний графік впровадження інфраструктури DevOps (діаграма Ганта)

ВИСНОВКИ

У звіті сформовано бачення інфраструктури DevOps для сервера «Розумного будинку» на Raspberry Pi 4 з підтримкою цифрового двійника (тема № 4 кафедри KIE).

1) Актуальність DevOps для проєкту зумовлена частими змінами в ході досліджень, розподіленою мікросервісною архітектурою, обмеженими ресурсами платформи та вимогами до відтворюваності наукових результатів; цільові показники визначено за метриками DORA (доставка зміни ≤ 30 хв, відновлення ≤ 15 хв).

2) Відповідальність розподілено між шістьма ролями за матрицею RACI для 13 видів діяльності; поєднання ролей розробника та DevOps-інженера компенсовано обов'язковим переглядом коду іншим учасником.

3) Визначено 12 задач впровадження з перевірюваними критеріями у чотирьох напрямах: керування кодом, CI, CD, експлуатація.

4) Описано вісім стадій життєвого циклу, узгоджених із ISO/IEC/IEEE 12207, і три середовища (dev, staging, production) з однаковими образами контейнерів; незалежність оновлення сервера та датчиків забезпечено версіонованим контрактом MQTT.

5) Сплановано безкоштовний інструментальний стек, сумісний із ARM64: GitHub і GitHub Actions з власним runner на Raspberry Pi, Docker/Buildx, GHCR, Ansible, Trivy, Prometheus/Grafana/Loki, restic.

6) Очікувана трудомісткість за PERT – 232 год, плановий бюджет з резервом – 246 год за 10 тижнів; визначено критичний шлях впровадження.

Запропонована інфраструктура може бути повторно використана для інших тем кафедри, побудованих навколо «Розумного будинку» та ESP8266.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. DeBellis D., Storer K., Lewis A. et al. Accelerate State of DevOps Report 2024. DORA, Google Cloud, 2024. URL: https://dora.dev/research/2024/dora-report/ (дата звернення: 06.11.2026).
2. Forsgren N., Humble J., Kim G. Accelerate: The Science of Lean Software and DevOps. Portland : IT Revolution Press, 2018. 288 p.
3. Мардєєва Д. Р., Вадурін К. О., Перекрест І. А. Розробка структури Розумного будинку з використанням технології Industry 5.0. *Актуальні проблеми життєдіяльності суспільства* : матеріали XXX Міжнар. наук.-практ. конф. студентів, аспірантів та молодих учених КрНУ ім. М. Остроградського, 20–21 квітня 2023 р. Кременчук, 2023. С. 71–72.
4. Kim G., Humble J., Debois P., Willis J., Forsgren N. The DevOps Handbook. 2nd ed. Portland : IT Revolution Press, 2021. 480 p.
5. ISO/IEC/IEEE 12207:2017. Systems and software engineering – Software life cycle processes. Geneva : ISO, 2017. 145 p. URL: https://www.iso.org/standard/63712.html (дата звернення: 06.11.2026).
6. Eclipse Mosquitto: An open source MQTT broker. URL: https://mosquitto.org/ (дата звернення: 06.11.2026).
7. About self-hosted runners. *GitHub Docs*. URL: https://docs.github.com/en/actions/hosting-your-own-runners (дата звернення: 06.11.2026).
8. Multi-platform builds. *Docker Docs*. URL: https://docs.docker.com/build/building/multi-platform/ (дата звернення: 06.11.2026).
9. Ansible Documentation. *Red Hat*. URL: https://docs.ansible.com/ (дата звернення: 06.11.2026).
10. Prometheus: Overview. URL: https://prometheus.io/docs/introduction/overview/ (дата звернення: 06.11.2026).

11. Humble J., Farley D. Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Boston : Addison-Wesley, 2010. 512 p.

12. Raspberry Pi Documentation: Raspberry Pi OS. URL: https://www.raspberrypi.com/documentation/computers/os.html (дата звернення: 06.11.2026).